

Detektory objektov

26.11.2025, 15:00-17:00

Andrej Lúčny

Premena klasifikátora na detektor. Stavebné prvky umožňujúce trénovanie hlbokých neurónových sietí: vynechávanie neurónov, normalizácia dávky, reziduálne spojenia. YOLO detektor (YOLO v3). Regiónové konvolučné siete (Faster RCNN)

<https://github.com/andy1ucny/OAI>

Úloha detekcie



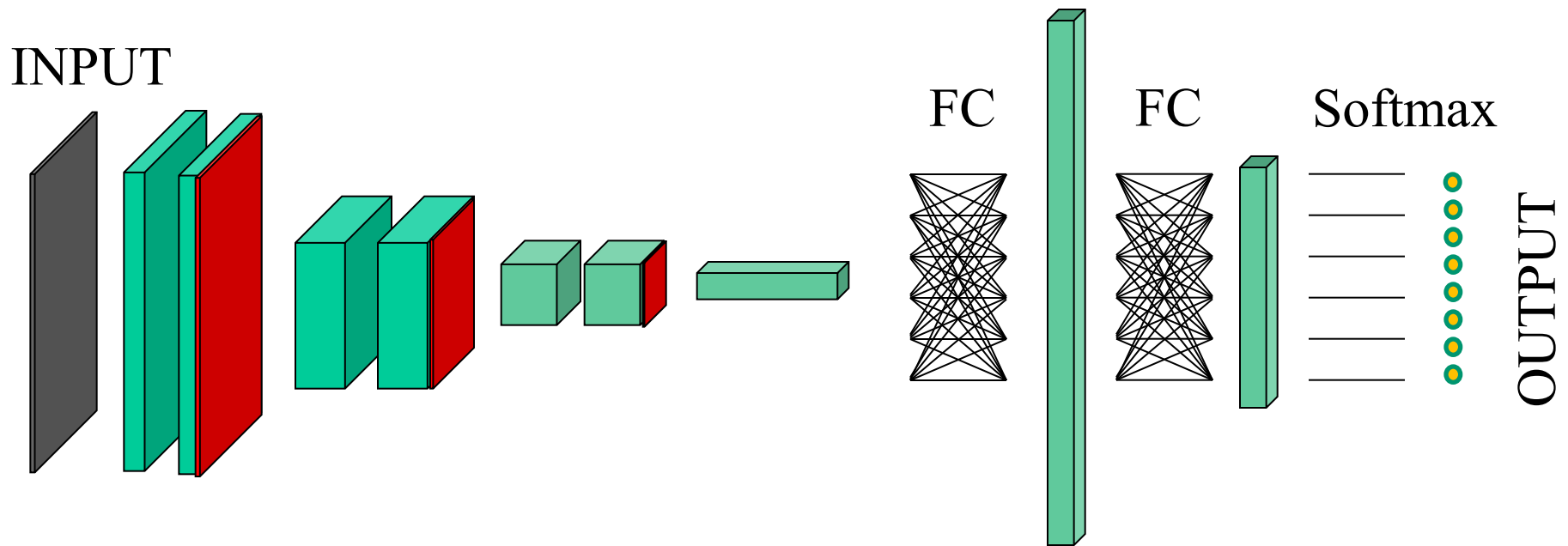
Úloha detekcie



Dátová sada: tréningová / testová

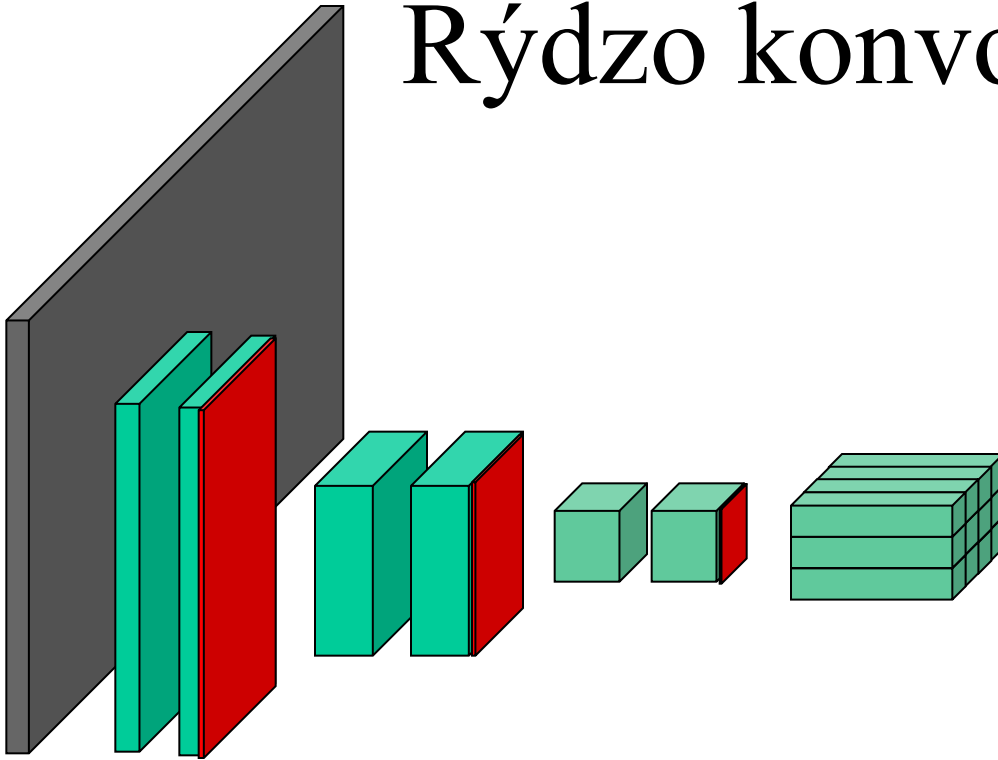


Klasifikátor



Minule sme sa dozvedeli ako urobiť klasifikátor

Rýdzo konvolučný backbone



- Vieme už aj to, že ak do chrbticová sieť dáva v princípe mapu príznakov.
- Pokiaľ ju krmíme obrázkom s väčším rozlíšením, zväčšuje sa aj rozlíšenie tejto mapy

Algoritmus plávajúceho okna (Sliding window algorithm)

Každý klasifikátor sa dá premeniť hrubou silou na detektor pomocou algoritmu používajúceho plávajúce okno:

- otestujeme všetky možné výskyty objektu



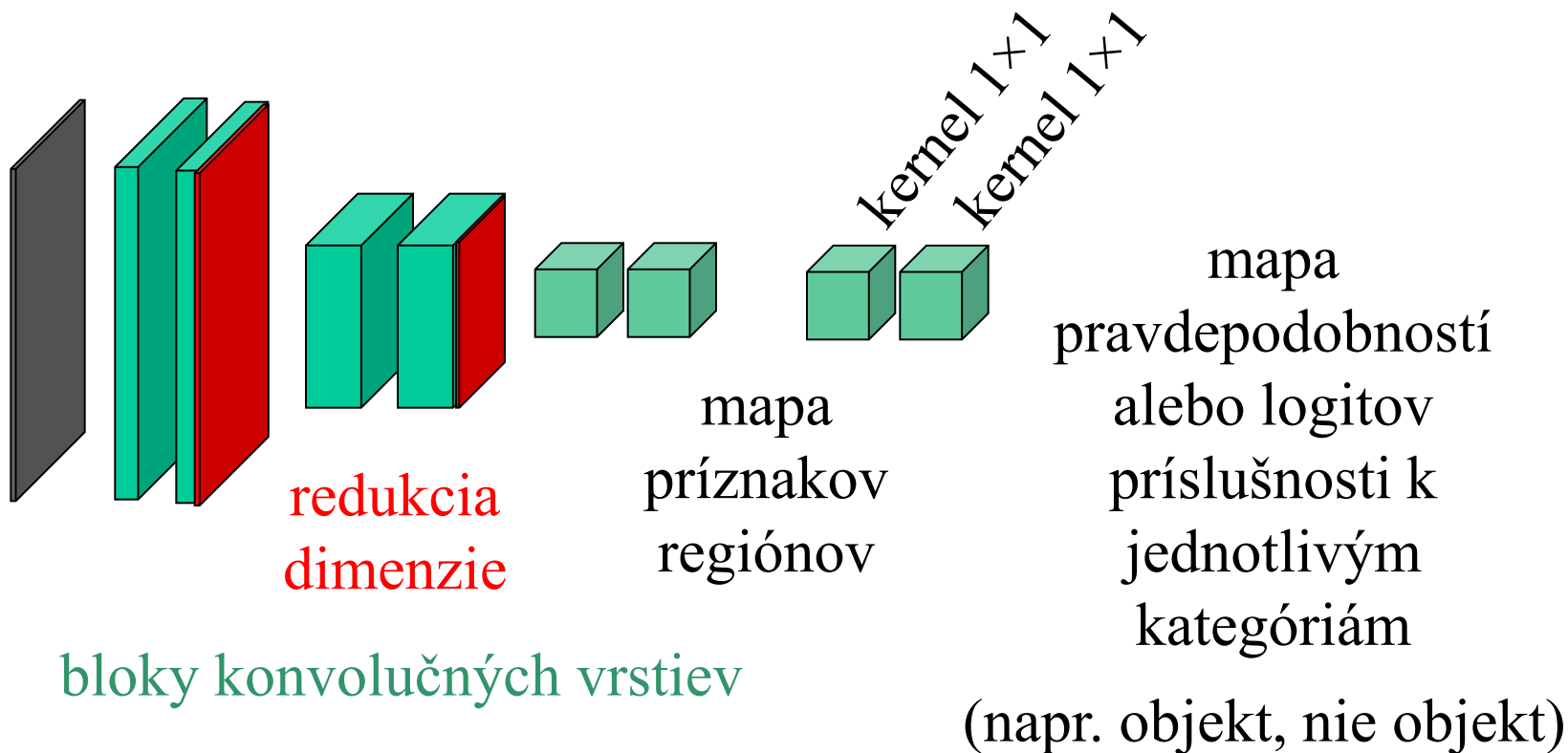
Detektor

Urobíme ho ako klasifikátor ktorý sa bude pozerat' na rôzne miesta na obraze a klasifikovat' či sú hľadaným objektom

Implementujeme ho ako perceptron, ktorý sa paralelne pozerá na všetky tieto miesta

To vieme urobiť pomocou dvoch či viacerých blokov konvolučných vrstiev

(jednoduchý) Detektor



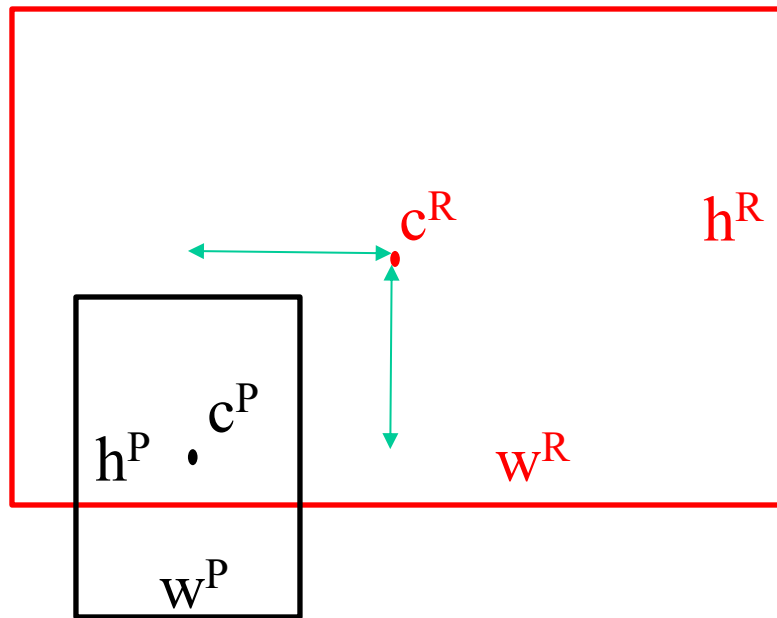
Nedostatky

- rôzna veľkosť objektov
- chýba spolupráca medzi pohľadmi na blízke regióny (ak tu vidím číslicu 5, aj kúsok vedľa by som mal vidieť 5)
- jeden objekt uvidím viac krát pričom jeho ohraničujúce obdĺžniky sa budú prekrývať
- fantomické detekcie

Riešenia: rôzna veľkosť objektov

- Budeme robiť nielen klasifikáciu ale aj regresiu objekt zahrňujúceho obdĺžnika

ukotvenie (anchor)



logity

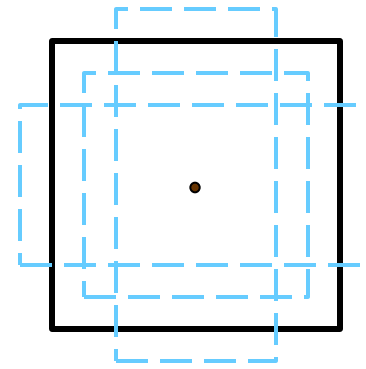
$$\left(\underbrace{\frac{c_x^R - c_x^P}{w^P}}_{b_0}, \underbrace{\frac{c_y^R - c_y^P}{h^P}}_{b_1}, \ln \underbrace{\frac{w^P}{w^R}}_{b_2}, \ln \underbrace{\frac{h^P}{h^R}}_{b_3} \right)$$

↓

$$\left(\underbrace{c_x^R - b_0 w^P}_{\parallel c_x^P}, \underbrace{c_y^R - b_1 h^P}_{\parallel c_y^P}, \underbrace{w^R e^{b_2}}_{\parallel w^P}, \underbrace{h^R e^{b_3}}_{\parallel h^P} \right)$$

pixely

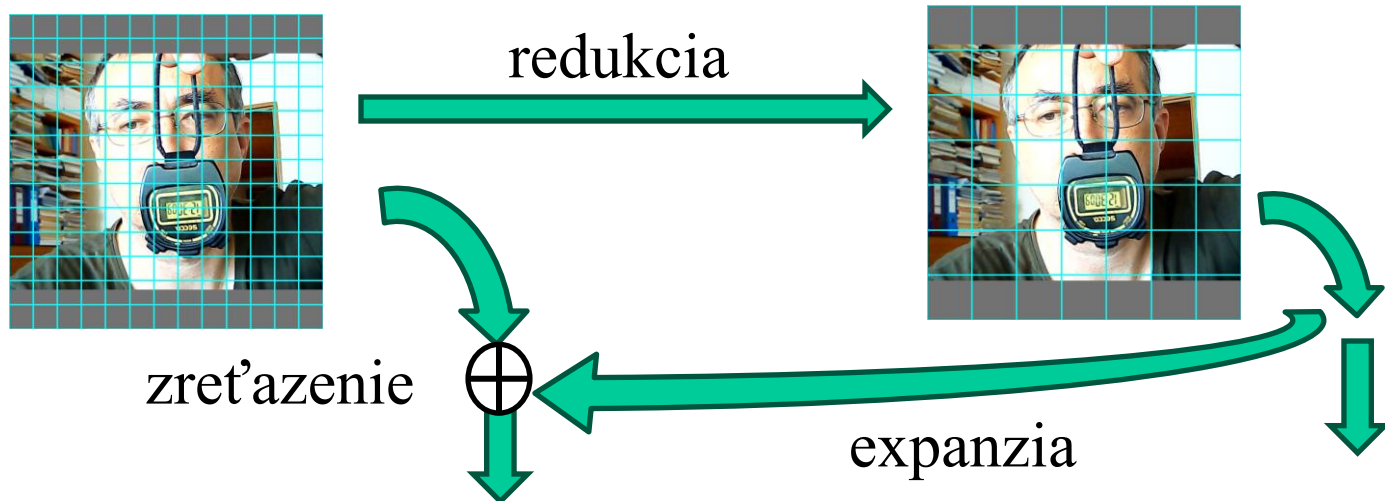
Ukotvenia



- Každá detekcia vyjadruje x, y, w , a h relatívne k určitému ukotveniu. YOLO v3 využíva tri ukotvenia pre každú z troch veľkostí objektov (dohromady deväť), každé s rôznym pomerom strán
- Ukotvenia v YOLO v3:
[116x90, 156x198, 373x326] (výstup 52x52)
[30x61, 62x45, 59x119] (výstup 26x26)
[10x13, 16x30, 33x23] (výstup 13x13)

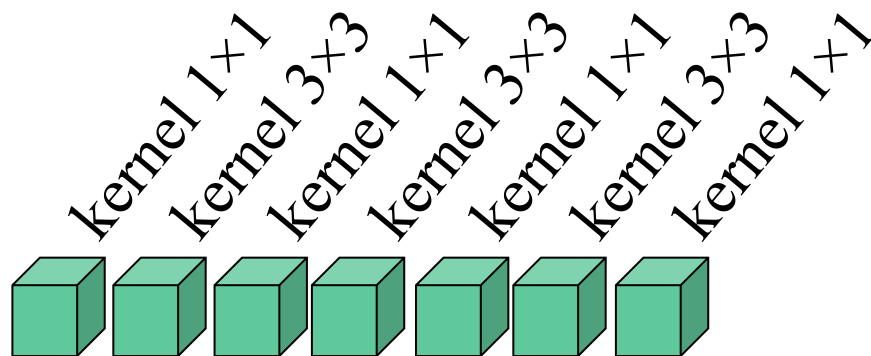
Riešenia: rôzna veľkosť objektov

- Budeme používať viacero príznakových máp s rôznym rozlíšením
- Predbežnými výsledkami detekcie pre podrobnejšie mapy príznakov budeme obohacovať



Riešenia: spolupráca blízkyh

- Konvolučné vrstvy s kernelmi 1×1 budeme prekladať vrstvami s kernelmi 3×3



Riešenia: jeden vidím viac krát

Potlačenie nemaximálnych hodnôt (Non-Maximum Suppression)

- Vezmeme detekciu s najväčšou spoľahlivosťou a vyhodíme všetky, ktoré sa s ňou podstatne prekrývajú

Riešenia: fantomické detekcie

- Klasifikujeme objektovosť, spoľahlivosť (objectness, confidence)
- V NMS berieme len detekcie so spoľahlivosťou nad určitý prah

Single-class verzus Multi-class

- Ak sa kategórie objektov vylučujú, logity meníme na pravdepodobnosti cez Softmax
- Ak to tak však nie je (máme napríklad kategóriu koleso i bicykel), používame na výstupe sigmoidu

YOLO Detektor (You Look Only Once)

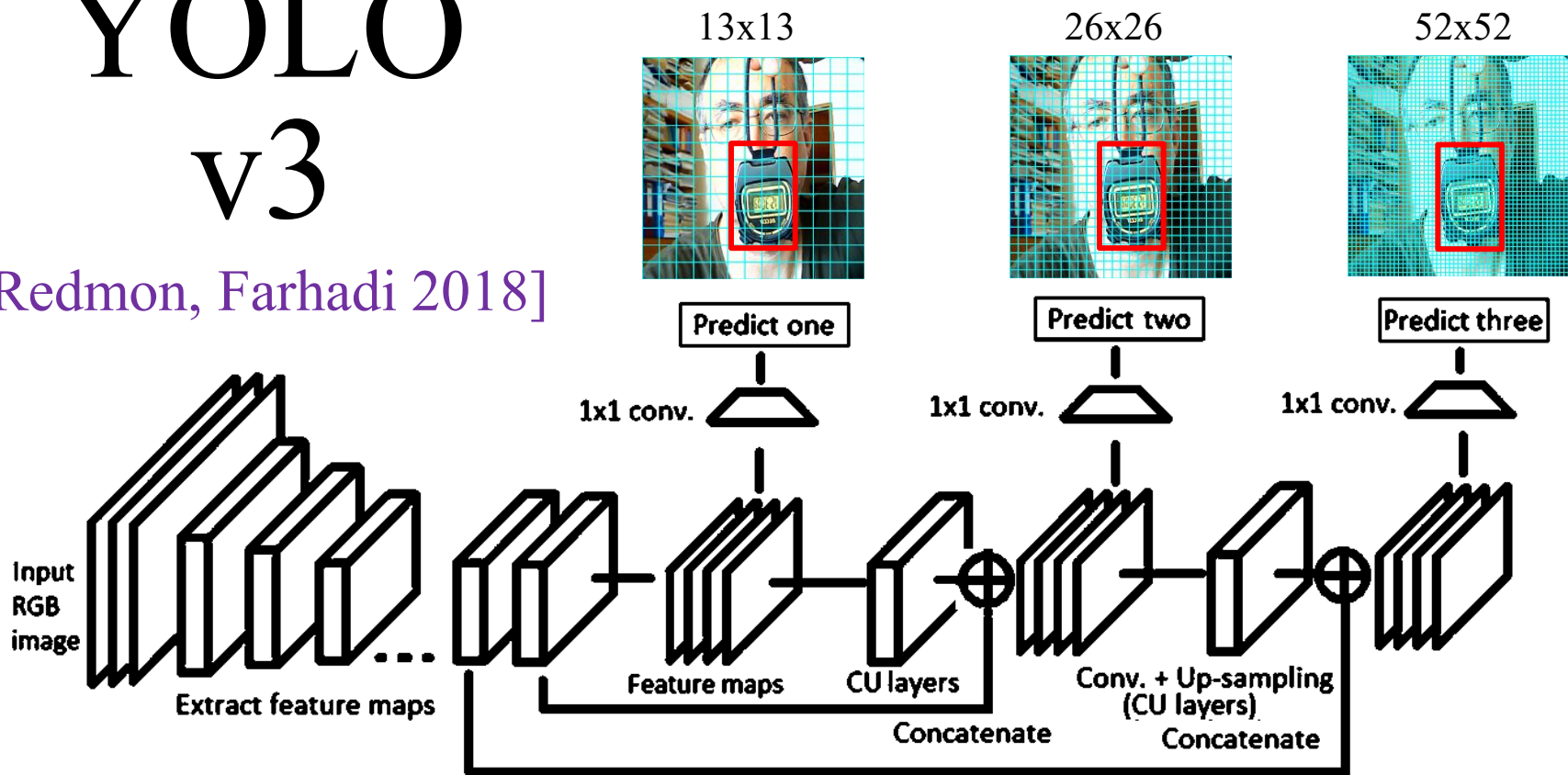
- obraz pokryjeme neprekrývajúcimi sa regiónmi
- nad každým regiónom vykonáme klasifikáciu a regresiu a zosumarizujeme výsledky



- **Klasifikátor** dáva pravdepodobnosť, že región patrí k detekovanému objektu
- **Regressor** dáva obdĺžnik obsahujúci objekt, ktorého je región súčasťou

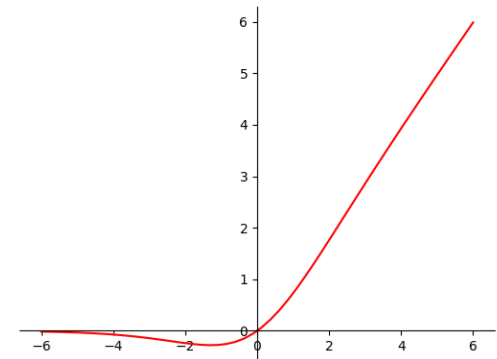
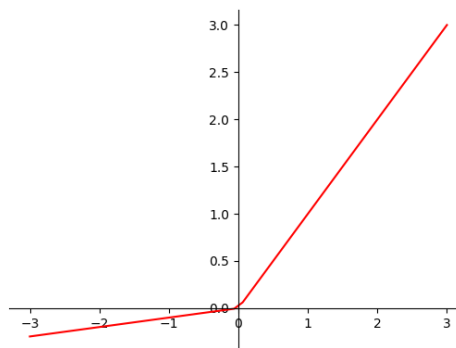
YOLO v3

[Redmon, Farhadi 2018]



252 blokov, 5219 vrstiev, max. hĺbka cca 200

Aktivácia:
LeakyRelu alebo SiLU



YOLO v3 - výstup

- $13 \times 13 \times 255$, $26 \times 26 \times 255$, $52 \times 52 \times 255$
- Každý perceptron dáva tri detekcie (pre tri kotvy), $3 \times 85 = 255$ hodnôt
- Každá detekcia dáva 85 hodnôt = $4+1+80$ obsahujúcich:
 - 4 ... kód ohraničujúceho obdĺžnika
 - 1 ... spoľahlivosť (confidence)
 - 80 ... pravdepodobnosť príslušnosti k jednej z 80 kategórii

Transférové učenie

YOLO v3 poskytuje predtrénovaný model pre 80 kategórii z dátovej vzorky COCO:

person, bicycle, car, motorbike, aeroplane, bus, train, truck, boat, traffic light, fire hydrant, stop sign, parking meter, bench, bird, cat, dog, horse, sheep, cow, elephant, bear, zebra, giraffe, backpack, umbrella, handbag, tie, suitcase, frisbee, skis, snowboard, sports, ball, kite, baseball bat, baseball glove, skateboard, surfboard, tennis racket, bottle, wine glass, cup, fork, knife, spoon, bowl, banana, apple, sandwich, orange, broccoli, carrot, hot dog, pizza, donut, cake, chair, sofa, pottedplant, bed, diningtable, toilet, tvmonitor, laptop, mouse, remote, keyboard, cell phone, microwave, oven, toaster, sink, refrigerator, book, clock, vase, scissors, teddy bear, hair drier, toothbrush

Pri trénovaní pre vlastné dátové vzorky, nezačínáme s náhodnými parametrami, ale parametrami predtrénovaného modelu.

Trénovanie hlbokých sietí

- Hlboké siete sa trénujú ťažko (problém miznúceho gradientu pri spätnom šírení)
- Preto do takýchto sietí vkladáme stavebné prvky, ktoré trénovanie podporujú.

Vynechávanie (Dropout)

$$drop(x) = \begin{cases} 0 & \text{ak } rnd < p \\ x \frac{1}{1-p} & \text{inak} \end{cases}$$

ak $p = 0.2$ 20% vynulujeme

80% zosilníme

Do siete vkladáme Dropouty

Normalizácia dávky (Batch normalization)

trénovanie $bn(x) = \gamma \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$

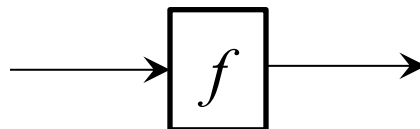
inferencia $bn(x) = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$

$$\mu = \sum_{i=1}^t (1 - \alpha)^{t-i} \mu_{B_i}$$

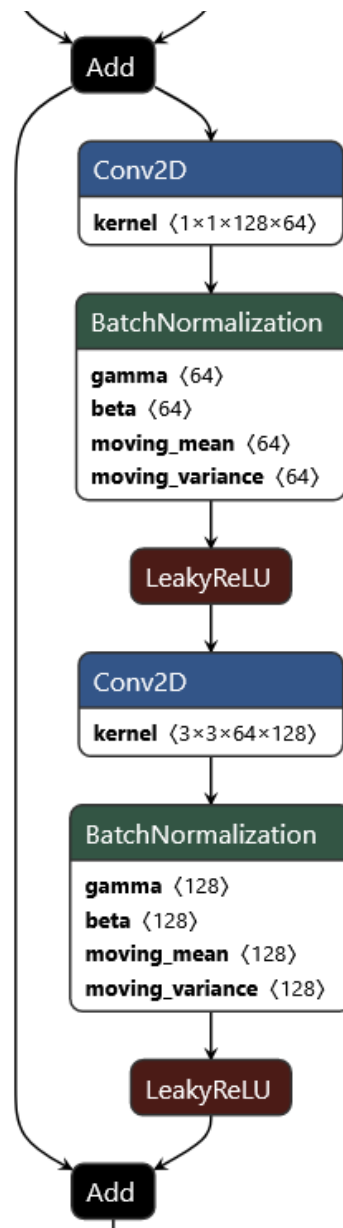
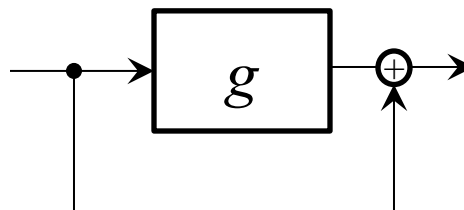
$$\sigma^2 = \sum_{i=1}^t (1 - \alpha)^{t-i} \sigma_{B_i}^2$$

Reziduálne spojenia (Residual connections)

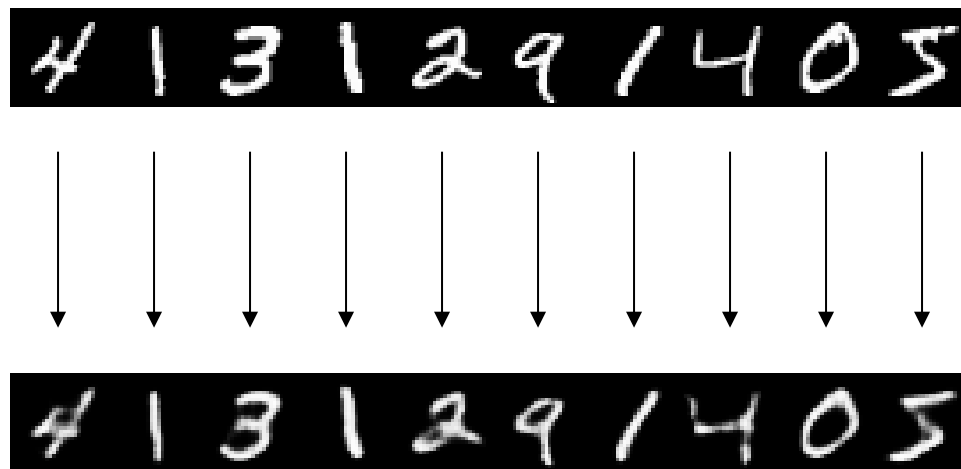
Nehl'adáme $f(x)$



Ale doplnok
(residuum) $g := f(x) - x$



- Vplyv týchto stavebných prvkov dobre vidno na tréňovaní identity
- Na rozdiel od autokódera však neredukujeme dimenziu





conv



conv

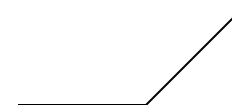


conv



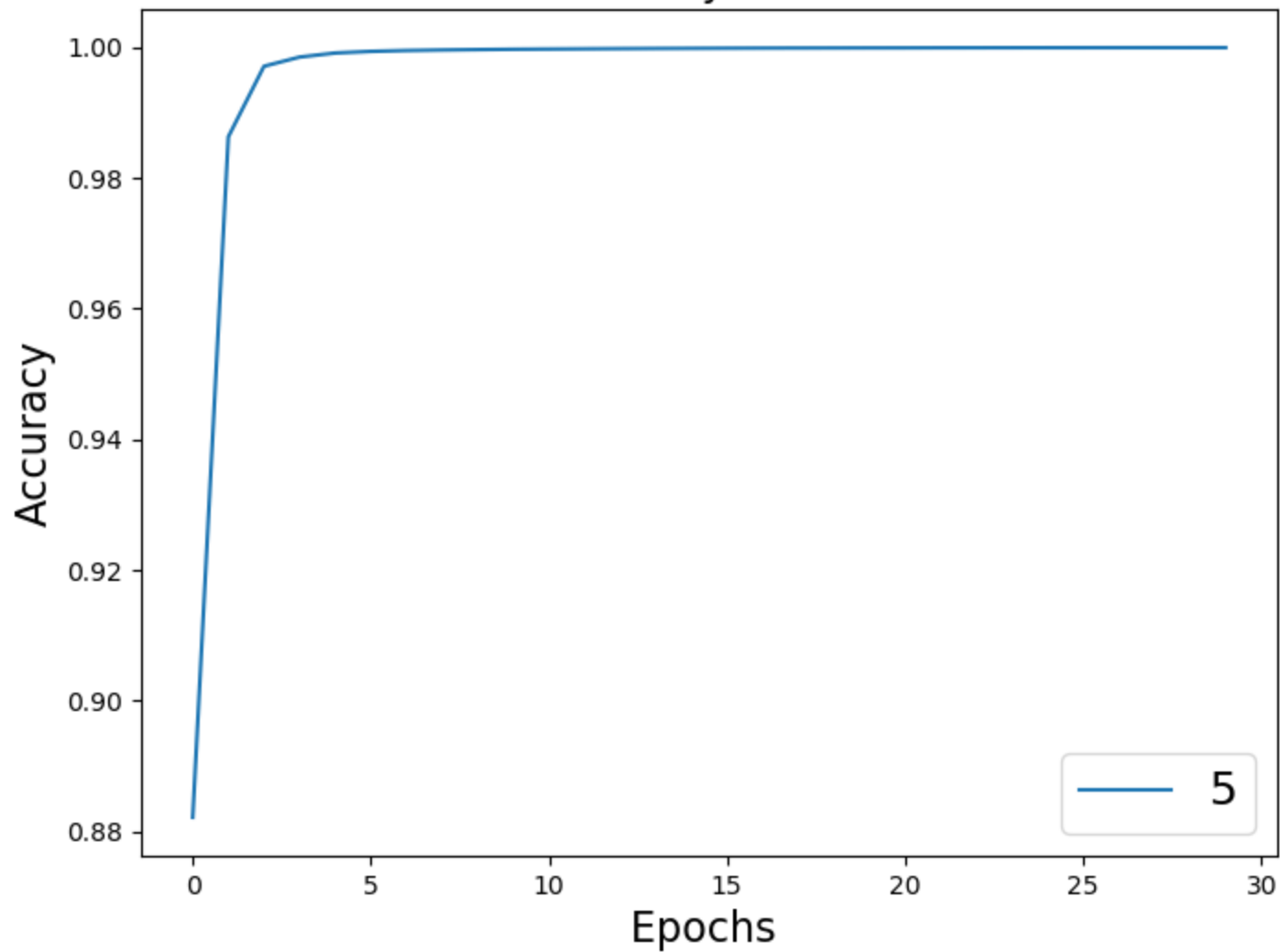
conv

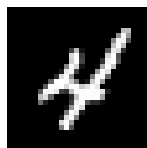
⋮



všade: ReLU

Accuracy Curves





conv



dropout

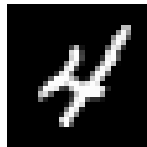


conv

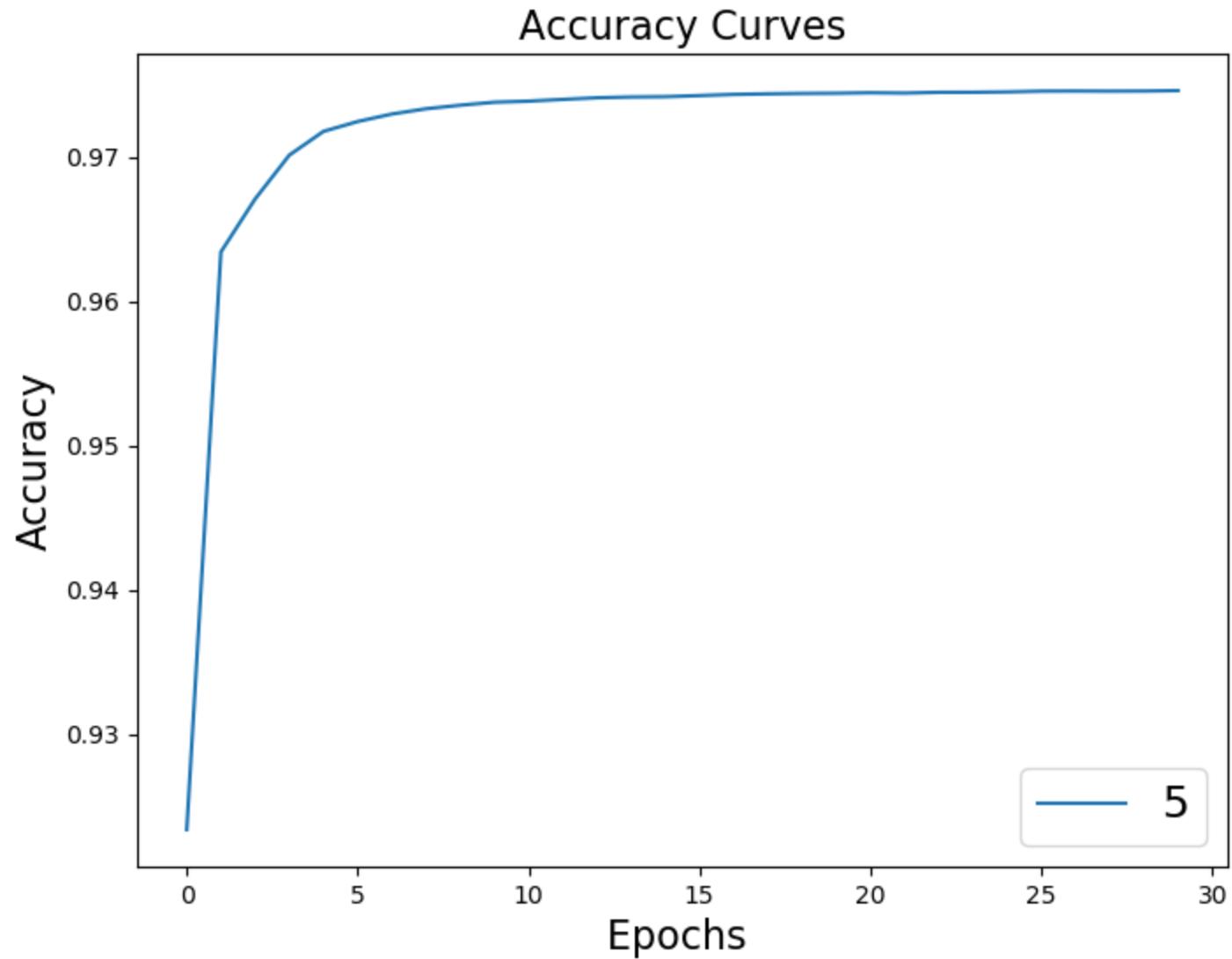


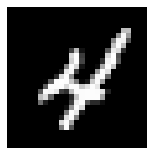
dropout

⋮



- Dropout





conv



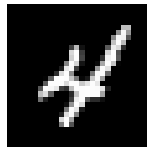
Batch normalization



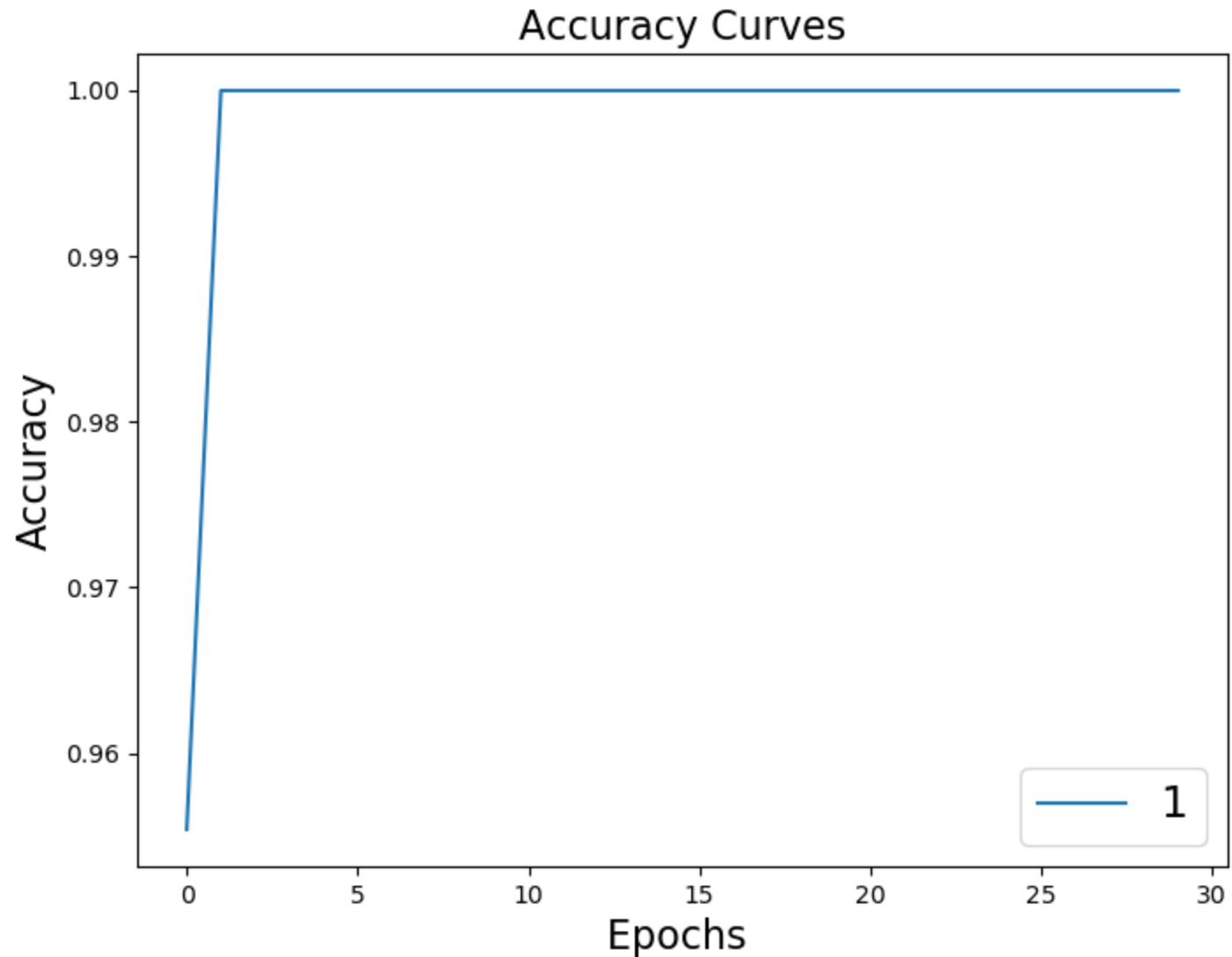
conv

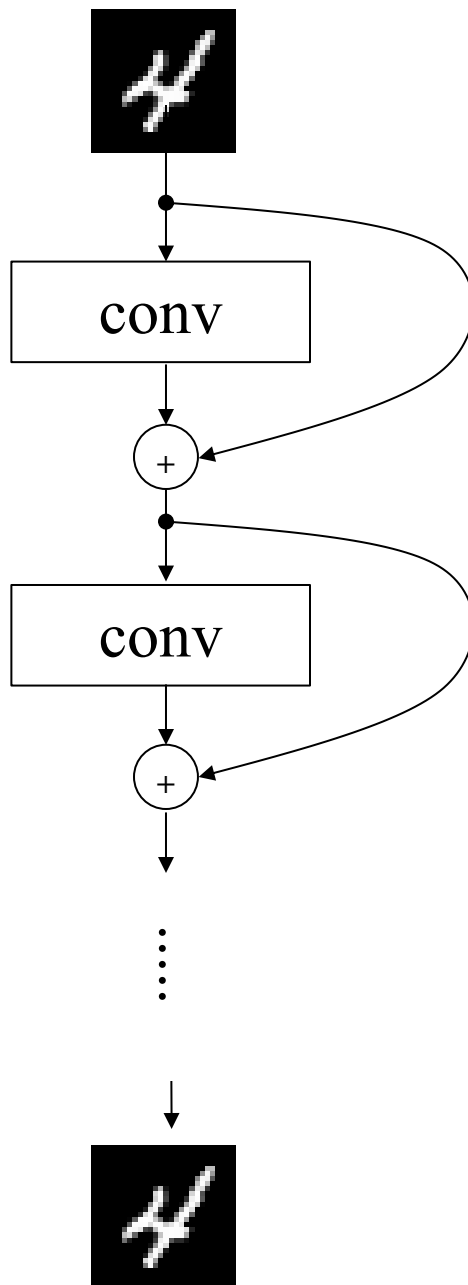


Batch normalization

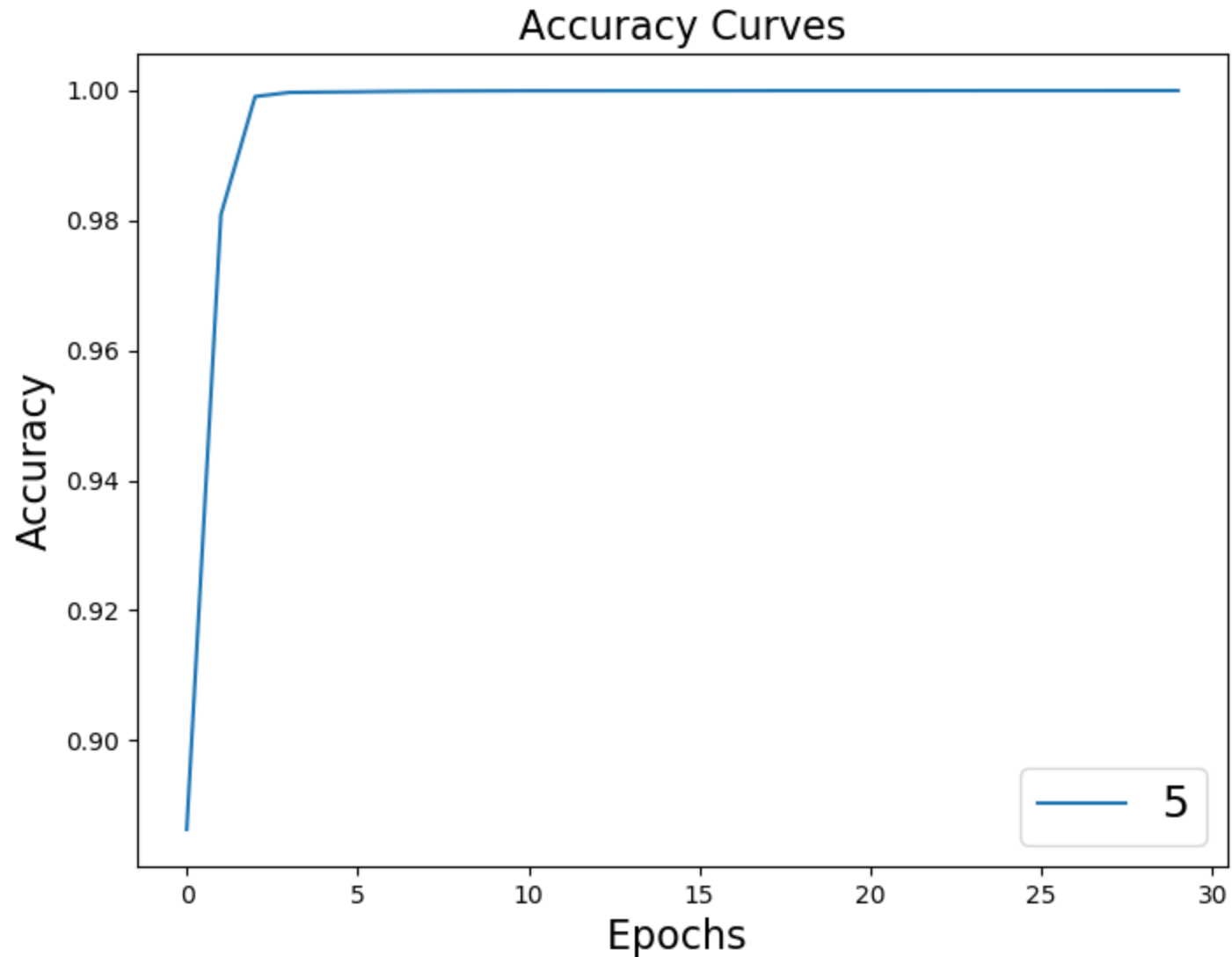


- Batch normalization





- Reziduály



Augmentácia (Augmentation)

- Anotovať dataset je namáhavé, takže hľadáme cesty ako príklady z datasety rozmnožiť.
- Napríklad v datasete tvárí, nám vertikálny flip zdvojnásobí dataset, lebo tváre nie sú úplne súmerné
- Operácie ktoré vyrábajú z jednej vzorky ďalšie (zväčšením, zmenšením, pootočením, posunutím, prevrátením, prefarbením, ...) nazývame augmentáciou

Trénovanie YOLO v3 <https://youtu.be/CPGR26Fhcro>



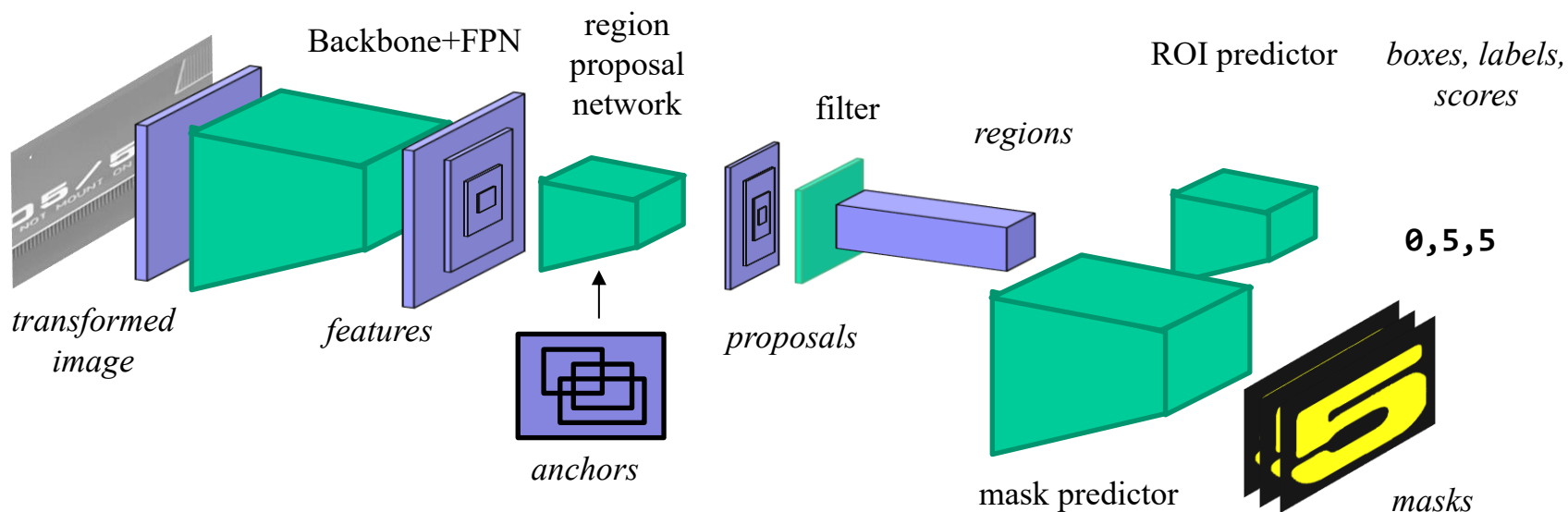
<https://youtu.be/ENYEUUbqjIE>

Spracovanie videa



Regiónové konvolučné siete

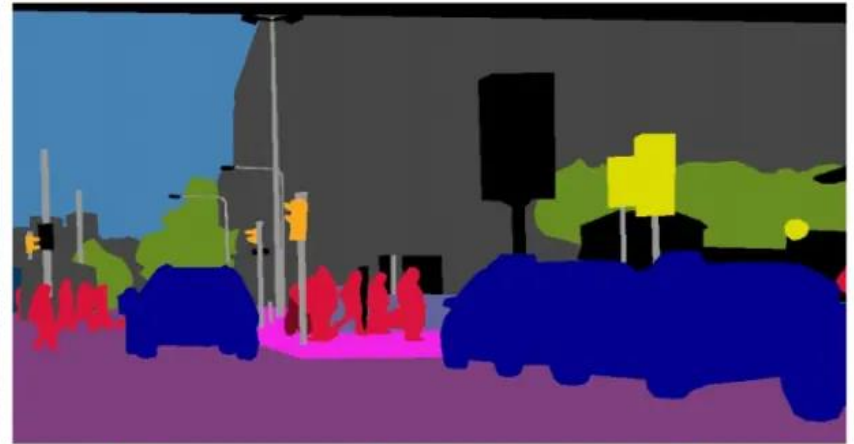
- Faster RCNN, MaskRCNN



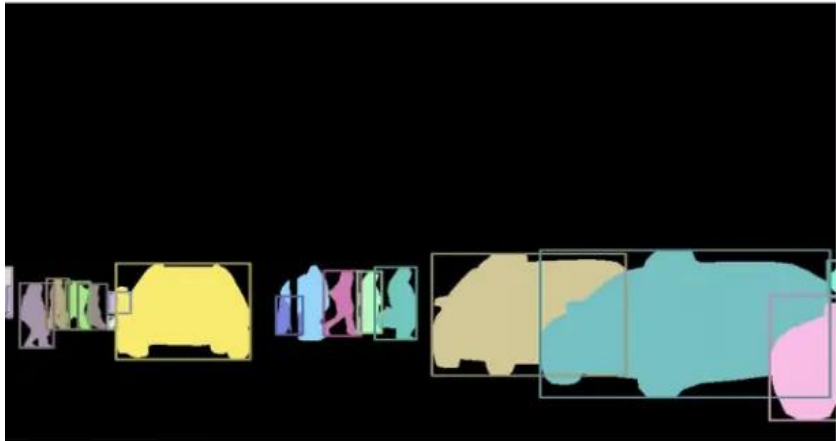
Segmentation



(a) image



(b) semantic segmentation



(c) instance segmentation



(d) panoptic segmentation